Lab 1 — Java Refresher and the Dispatch Board

Setup, primitives, arrays, loops, and `static`

Alex (Aizierjiang Aiersilan)
Lab Lead, Teaching Assistant

August 23, 2026

The George Washington University
CSCI 2113 - Software Engineering | Fall 2026

© Aizierjiang Aiersilan. Free distribution and open-access.

Outline

1. Getting Java Running
2. Primitive Types
3. Arrays and Loops
4. Static Members and References
5. The Assignment
6. Wrap-up
7. Accessibility Notice
8. Acknowledgement

ng Aiersilan | CSCI 2113

Getting Java Running

ng Aiersilan | CSCI 2113 | F

What we are doing in the next 35 minutes

Today is a **refresher**, not new material — but everything this semester is built on it.

- Get **javac** and **java** working on *your* machine
- Re-meet **primitive types**, **arrays**, **loops**, **static**
- Start the semester-long **Dispatch** project

💡 We build one thing all term: a small ride-sharing dispatcher. Today it is four arrays and a loop.

Clock

0–6	setup, first compile
6–11	primitives, overflow
11–16	arrays and loops
16–26	`static`, references
26–33	the assignment
33–35	questions

⚠ Code along. Don't just watch.

Aierqian / CSU 2113

Your toolchain: two commands, one rule

You need a **JDK** (Java Development Kit). A **JRE** only *runs* Java — it has no compiler.

1. **javac DispatchBoard.java** → produces `DispatchBoard.class` (**bytecode**)
2. **java DispatchBoard** → the **JVM** runs that bytecode

⚠ Step 2 takes the *class name*, never a file name: no `.java`, no `.class`.

Check what you have with `java -version` and `javac -version`. Anything from **JDK 17** onward behaves identically for this whole lab.

In VS Code

- Install **Extension Pack for Java** (Microsoft)
- ✔ Open the *folder*, not the lone file — the language server needs a project root
- Terminal: `Ctrl+‘`
- A `Run` link appears above `main`

The one rule

A `public class` must live in a file of exactly the same name.

Activity 1 — your first compile

Create **DispatchBoard.java**:

```
public class DispatchBoard {
    public static void main(String[] args) {
        System.out.println("Dispatch board online.");
    }
}
```

Then, in the folder that holds the file:

```
$ javac DispatchBoard.java
$ java DispatchBoard
Dispatch board online.
```

✔ `ls` (or `dir`) now shows a `.class` file.

💡 Since **JDK 11** you may also run `java DispatchBoard.java` in one step [4] — but know the two-step model first, because `javac` is where errors surface.

Reading the signature: `public` = visible everywhere; `static` = callable with no object; `void` = returns nothing; `String[] args` = the words typed after the class name.

The four errors you will actually see

What the tool says	Why	Fix
`class Dispatchboard is public, should be declared in a file named Dispatchboard.java`	File name ≠ public class name; Java is case-sensitive	Rename one so they match
`Error: Could not find or load main class dispatchboard`	Wrong capitals, wrong folder, or you typed `.class`	`cd` to the folder holding the `.class`, then `java DispatchBoard`
`'javac' is not recognized command not found: javac`	No JDK on your `PATH`	Install a JDK, then open a new terminal
`error: ';' expected`	A missing semicolon, usually on the line *above* the caret	Go to that line number; the ^ marks the spot

💡 Always fix the topmost error and recompile — one real mistake often prints five messages.

Primitive Types

ng Aiersilan | CSCI 2113 | F

The eight primitive types

Type	Size	Holds	Literal
`byte`	**8** bits	$-128 \ldots 127$	`4`
`short`	**16** bits	$-32{,}768 \ldots 32{,}767$	`4`
`int`	**32** bits	$-2{,}147{,}483{,}648 \ldots 2{,}147{,}483{,}647$	`4`
`long`	**64** bits	about $\pm 9.2 \times 10^{18}$	`4L`
`float`	**32** bits	$\approx$ **7** significant digits	`4.8f`
`double`	**64** bits	$\approx$ **15–16** significant digits	`4.8`
`char`	**16** bits	one UTF-16 unit, $0 \ldots 65{,}535$	`'B'`
`boolean`	1 bit of information	`true` or `false`	`true`

⚠ A bare `4` is an **int**; a bare `4.8` is a **double**. That is why the highlighted rows are the two you use daily.

💡 A primitive **holds its value**. Everything else — `String`, arrays — holds a **reference**. That comes back in ten minutes.

Activity 2 — declare, print, and read carefully

Add this inside `main`:

```
int     seats     = 4;
double  rating    = 4.8;
boolean available = true;
char    zoneCode  = 'B';

System.out.println(seats + " seats, rated "
    + rating + ", zone " + zoneCode);
System.out.println("Available? " + available);

int big = 2147483647;   // Integer.MAX_VALUE
System.out.println(big + 1);
```

The shape never changes: `type name = value;`

What actually prints:

```
4 seats, rated 4.8, zone B
Available? true
-2147483648
```

The first two lines are unremarkable — `println` happily turns a `double`, a `boolean` and a `char` into text.

⚠ The third line is not **2147483648**. Nothing crashed, and no warning was printed.

Why `2147483647 + 1` is negative

An `int` is exactly **32** bits in two's complement, so it can hold 2^{32} distinct values — no more.

- `Integer.MAX_VALUE` = **2147483647**
- `Integer.MIN_VALUE` = **-2147483648**
- Add one to the top and you land on the bottom: it is an **odometer**, not a ladder.

⚠ Java never reports integer overflow [3]. Your program keeps running with a wrong number — which is exactly what makes this bug expensive.

If you need bigger

- Use `long`: `2147483647L + 1L` gives **2147483648**
- Or ask to be told: `Math.addExact(a, b)` throws `ArithmeticException: integer overflow`

💡 Same trap, everyday version: a counter of milliseconds overflows an `int` after about **25** days.

Two operators that do not mean what you think

+ has two jobs

```
1 + 2 + " rides"  -> "3 rides"
"rides: " + 1 + 2 -> "rides: 12"
'B' + 1           -> 67
"" + 'B' + 1      -> "B1"
```

+ works **left to right**. If *either* side is a `String` it joins text; otherwise it adds numbers — and a `char` is promoted to its numeric code first (`'B'` is **66**).

✔ When in doubt, add parentheses.

/ depends on the types

```
49 / 5    -> 9     // int / int
49 / 5.0  -> 9.8   // one double
```

Two `int`s divide to an `int`: the remainder is **discarded**, not rounded. Make one side a `double` to get a real average.

⚠ And `double` is binary, so `0.1 + 0.2` prints **0.30000000000000004** [2]. Never compare two `double`s with `==`; print with `%.2f` instead.

Arrays and Loops

ng Aiersilan | CSCI 2113 | F

Anatomy of an array

```
String[] driverNames =
    {"Ada", "Grace", "Linus", "Barbara", "Alan"};

int[] counts = new int[5];   // all zeros
```

- Indices run **0** to `length - 1` — here **0** to **4**
- Size is **fixed** the moment the array is created
- `new` fills with defaults: **0**, **0.0**, `false`, `null`
- An array is an **object**: the variable holds a reference

The parenthesis trap

`driverNames.length` — an array's length is a **field**: no parentheses.

`driverNames[0].length()` — a `String`'s length is a **method**: parentheses required.

Step one index too far and you get:

```
ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 5
```

✔ The message names both numbers — read it, don't guess.

Activity 3 — print every driver

Indexed `for` — you get `i`

```
for (int i = 0; i < driverNames.length; i++) {
    System.out.println(driverNames[i]);
}
```

Three parts: **start**; **keep going while**; **step**. The same loop as a `while`:

```
int i = 0;
while (i < driverNames.length) {
    System.out.println(driverNames[i]);
    i++;
}
```

Enhanced `for` — you get the value

```
for (String name : driverNames) {
    System.out.println(name);
}
```

Reads as "for each `name` in `driverNames`". Shorter, and it cannot go out of bounds.

Which one?

✔ Only need the value → enhanced `for`.
✔ Need the position → indexed `for`.
We will need the position in **60** seconds, to line up *two* arrays.

⚠ Writing `i <= driverNames.length` is the single most common bug in this lab.

Static Members and References

ng Aiersilan | CSCI 2113 | F

`static`, in one slide

```
public class DispatchBoard {
    static int totalRides = 0;      // class variable

    public static void main(String[] args) {
        int seats = 4;              // local variable
    }
}
```

- **static** = belongs to the *class*. One copy, created when the class loads, alive with no object in sight.
- Non-static (**instance**) members need an object — we have no objects yet, so today everything is `static`.
- `main` is `static` because the JVM must call it *before* any object exists.

Scope

A **local** variable lives only inside its `{ }` block, and you must assign it before reading it.

A **static field** is visible to every method in the class and starts at its default (**0**, `false`, `null`).

Mix them up and `javac` says:

```
error: non-static variable seats
cannot be referenced from a
static context
```

✔ Translation: "that thing needs an object; `main` has none."

Activity 4 — one loop, two arrays

```
static int totalRides = 0;

public static void main(String[] args) {
    String[] driverNames =
        {"Ada","Grace","Linus","Barbara","Alan"};
    int[]    ridesToday  =
        {   12,      8,     15,        5,     9};

    for (int i = 0; i < ridesToday.length; i++) {
        totalRides += ridesToday[i];
    }
    System.out.println("Total rides today: "
                       + totalRides);
}
```

```
Total rides today: 49
```

The accumulator pattern

1. Create the total **outside** the loop
2. Update it **inside** the loop
3. Use it **after** the loop

`totalRides += x` is shorthand for `totalRides = totalRides + x`.

⚠ Declare the total *inside* the loop and it resets to **0** on every pass — you will print **9**, the last driver's rides.

💡 Index `i` means the same driver in both arrays. That idea is the whole assignment.

Activity 5a — one array, two names

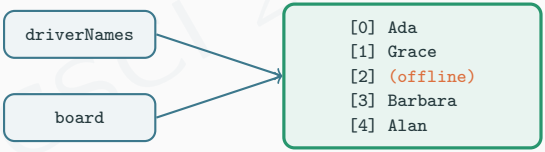
```
String[] board = driverNames;
board[2] = "(offline)";
System.out.println(driverNames[2]);
```

```
(offline)
```

We never touched `driverNames`. So why did it change?

Because = on an array copies the **arrow**, not the boxes. Both names now point at one single array.

✔ Want a real copy? `driverNames.clone()` or `Arrays.copyOf(driverNames, driverNames.length)`.



two references, one object

💡 With primitives there is no arrow:
`int a = 5; int b = a; b = 9;` leaves `a` at **5**.

Activity 5b — `takeOffline`, and what a method can change

```
static void takeOffline(String[] arr, int index) {
    arr[index] = "(offline)";
}

// in main:
takeOffline(driverNames, 2);
System.out.println(driverNames[2]);
```

```
(offline)
```

A `static` method needs no object, so `main` can call it directly. It takes the array to work on and the index to change — so it works for *any* array, not just this one.

Java is always pass-by-value

The *value* copied for an array is the arrow.

- ✔ `arr[i] = ...` follows the arrow → the caller sees the change
- ✖ `arr = new String[]{...}` re-points only the local copy → caller sees nothing
- ✖ `n = n + 100` on an `int` parameter → caller sees nothing

💡 Rule of thumb: a method can **edit** what you hand it; it cannot **replace** it.

Activity 6 — the busiest driver

```
static String findBusiestDriver(String[] names,
                                int[] rides) {
    int best = 0;    // an index, not a value
    for (int i = 1; i < rides.length; i++) {
        if (rides[i] > rides[best]) {
            best = i;
        }
    }
    return names[best];
}
```

```
Busiest Driver: Linus
```

Track the index

Keeping `int max = 15` would tell you the *number* of rides — but the answer you need lives in the **other** array. The index is the only thing that links them.

Driver **2** has **15** rides, the most of **12, 8, 15, 5, 9**.

⚠ The handout calls it `busiestDriver()` in the heading and `findBusiestDriver()` in the code. Pick one — the name at the call site must match the declaration exactly, or you get `error: cannot find symbol`.

💡 What should this return for an *empty* array? Hold that thought.

The Assignment

ng Aiersilan | CSCI 2113 | F

Dispatch.java — four parallel arrays

i	names	zones	available	ratings
0	Ada	3	true	4.8
1	Grace	1	false	5.0
2	Linus	4	true	4.1
3	Ken	1	true	4.6
4	Barbara	2	false	4.9
5	Alan	4	true	4.4

Row `i` is one driver. Read across, and never sort one array on its own — that silently scrambles the other three.

Plus `int[][] distance`, where `distance[a][b]` is the travel time in minutes from zone `a` to zone `b`.

Six static methods

1. `void printBoard()`
2. `int countAvailable()`
3. `double averageRating()`
4. `double ratingOf(String name)`
5. `String bestRatedAvailable()`
6. `String nearestAvailable(int zone)`

Every one of them is a single loop over the arrays, carrying either a **running total** or a **best-so-far index**. You already wrote both patterns today.

printBoard() — lining up the columns

```
int width = 0;                  // pass 1
for (int i = 0; i < names.length; i++) {
    int len = names[i].length();
    if (len > width) width = len;
}
String fmt = "%-" + width;      // pass 2
fmt = fmt + "s | zone %d | %.1f | %s%n";
for (int i = 0; i < names.length; i++) {
    String status = "UNAVAILABLE";
    if (available[i]) status = "AVAILABLE";
    System.out.printf(fmt, names[i],
        zones[i], ratings[i], status);
}
```

```
Ada     | zone 3 | 4.8 | AVAILABLE
Grace   | zone 1 | 5.0 | UNAVAILABLE
Linus   | zone 4 | 4.1 | AVAILABLE
Ken     | zone 1 | 4.6 | AVAILABLE
Barbara | zone 2 | 4.9 | UNAVAILABLE
Alan    | zone 4 | 4.4 | AVAILABLE
```

printf format codes

%s	a string
%-7s	. . . left-justified in 7
%d	an int
%.1f	a double, 1 decimal
%n	newline (portable)

⚠ You need **two passes**: you cannot know the width until you have seen every name. Here it is 7 — "Barbara" — compute it, don't type 7.

✔ Building the format string with + is ordinary concatenation — the same operator we just looked at.

ratingOf() — the == trap

```
static double ratingOf(String driverName) {
    for (int i = 0; i < names.length; i++) {
        if (names[i].equals(driverName)) {
            return ratings[i];
        }
    }
    return -1.0;   // only after the whole loop
}
```

```
ratingOf("Grace")  ->  5.0
ratingOf("Turing") ->  -1.0
```

⚠ `return -1.0;` belongs **after** the loop. Put it in an `else` inside the loop and you give up after checking Ada.

Why == betrays you

== on objects asks *"same box?"*, not *"same contents?"* [1]

```
"Ada" == "Ada"           -> true
"Ada" == new String(..) -> false
"Ada".equals(new String(..))
                         -> true
```

The first line is `true` only because identical literals are shared in one pool — so the bug hides during testing and appears the moment a name is read from input.

✔ For text, always `.equals(...)` — or `.equalsIgnoreCase(...)`.

bestRatedAvailable() — filter, then maximise

```
static String bestRatedAvailable() {
    int best = -1;                    // "nobody yet"
    for (int i = 0; i < names.length; i++) {
        if (available[i] &&
            (best == -1 || ratings[i] > ratings[best])) {
            best = i;
        }
    }
    return best == -1 ? "NO DRIVER AVAILABLE"
                      : names[best];
}
```

```
bestRatedAvailable()  ->  Ada   (4.8)
```

Not Grace

Grace has the top rating in the file — **5.0** — and is `false` in `available`. If your answer is Grace, your filter never ran.

Seed with **-1**, not **0**: driver **0** might be unavailable, and then "best so far" would be a lie.

💡 `&&` and `||` **short-circuit**: because `best == -1` is tested first, `ratings[best]` is never evaluated while `best` is still **-1**. Order matters.

nearestAvailable(4) — a 2D lookup and a tie

```
int best = -1;
for (int i = 0; i < names.length; i++) {
    if (!available[i]) continue;
    if (best == -1) { best = i; continue; }
    int d  = distance[riderZone][zones[i]];
    int db = distance[riderZone][zones[best]];
    boolean closer = d < db;
    boolean tie    = (d == db)
        && (ratings[i] > ratings[best]);
    if (closer || tie) best = i;
}
return best == -1 ? "NO DRIVER AVAILABLE"
                  : names[best];
```

`distance` is an **array of arrays**: `distance.length` is **5** rows, `distance[0].length` is **5** columns. The **rider's** zone picks the row, the **driver's** zone picks the column.

Driver	zone	`distance[4][zone]`	rating
Ada	3	5	4.8
Linus	4	0	4.1
Ken	1	11	4.6
Alan	4	0	4.4

Grace and Barbara are skipped — unavailable. Linus and Alan tie at **0** minutes, so the higher rating wins: **Alan**.

⚠ A sloppy `d <= db` also prints Alan here — by luck, because he comes last. Handle the tie **explicitly** or you ship a bug that passes your own test.

Check yourself before you submit

Expected values

`countAvailable()`	4
`averageRating()`	4.6333... (4.63)
`ratingOf("Grace")`	5.0
`ratingOf("Turing")`	-1.0
`bestRatedAvailable()`	Ada
`nearestAvailable(0)`	Ken
`nearestAvailable(1)`	Ken
`nearestAvailable(2)`	Ada
`nearestAvailable(3)`	Ada
`nearestAvailable(4)`	Alan

The seven mistakes I will see

1. `==` instead of `.equals` on names
2. `.length()` on an array (or `.length` on a `String`)
3. `i <= names.length` in a loop bound
4. Dividing two `int`s and losing the decimals
5. Hard-coding **6** drivers or width **7**
6. `return`ing the "not found" value inside the loop
7. Forgetting the `available` filter entirely

✔ Run `printBoard()` first — it makes every later bug visible.

Wrap-up

ng Aiersilan | CSCI 2113 | F

Three things to keep

1. ★ **Two steps, one rule.** **javac** makes bytecode, **java** runs it — and a `public class` must match its file name exactly.
2. ★ **Values versus arrows.** A primitive holds its value and has hard edges (**2147483647** then **-2147483648**); an array variable holds a reference, so copying it copies the arrow, not the data.
3. ★ **One index, many arrays.** Parallel arrays stay in step through a shared `i` — and every method in the assignment is one loop carrying a running total or a best-so-far index.

In one sentence

Everything you will build this semester rests on
loop over an index, and know what your variable really holds.

Accessibility Notice

ng Aiersilan | CSCI 2113 | F

Accessibility Notice

Students who require DDA accommodations should contact the appropriate university support office and notify the instructional team as early as possible.

ng Aiersilan | CSCI 2113

Acknowledgement

ng Aiersilan | CSCI 2113 | F

Acknowledgement

This lab builds on the initial materials developed by **Timothy Kim**.

We gratefully acknowledge his contribution.

ng Aiersilan | CSCI 2113

Where to look things up

[1] J. Bloch.
Effective Java.
Addison-Wesley, 3rd edition, 2018.

[2] D. Goldberg.
What every computer scientist should know about floating-point arithmetic.
ACM Computing Surveys, 23(1):5–48, 1991.

[3] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman.
The Java Language Specification, Java SE 21 Edition.
Oracle America, Inc., 2023.

[4] OpenJDK.
JEP 330: Launch single-file source-code programs.
`https://openjdk.org/jeps/330`, 2018.

[5] Oracle.
The java tutorials: Language basics.
`https://docs.oracle.com/javase/tutorial/`.

💡 Start with the Oracle tutorial; the specification is the tie-breaker.

Questions?

Then start the assignment — I'm here for the rest of the hour.

Backup: `Dispatch.java` reference solution (1/2)

```
static void printBoard() {
    int width = 0;
    for (int i = 0; i < names.length; i++) {
        if (names[i].length() > width) width = names[i].length();
    }
    for (int i = 0; i < names.length; i++) {
        String status = available[i] ? "AVAILABLE" : "UNAVAILABLE";
        System.out.printf("%-" + width + "s | zone %d | %.1f | %s%n",
                          names[i], zones[i], ratings[i], status);
    }
}

static int countAvailable() {
    int count = 0;
    for (int i = 0; i < available.length; i++) {
        if (available[i]) count++;
    }
    return count;
}

static double averageRating() {
    double sum = 0.0;
    for (int i = 0; i < ratings.length; i++) sum += ratings[i];
    return sum / ratings.length;      // double / int -> double
}
```

Backup: `Dispatch.java` reference solution (2/2)

```
static double ratingOf(String driverName) {
    for (int i = 0; i < names.length; i++) {
        if (names[i].equals(driverName)) return ratings[i];
    }
    return -1.0;
}

static String bestRatedAvailable() {
    int best = -1;
    for (int i = 0; i < names.length; i++) {
        if (available[i] && (best == -1 || ratings[i] > ratings[best])) best = i;
    }
    return best == -1 ? "NO DRIVER AVAILABLE" : names[best];
}

static String nearestAvailable(int riderZone) {
    int best = -1;
    for (int i = 0; i < names.length; i++) {
        if (!available[i]) continue;
        if (best == -1) { best = i; continue; }
        int d     = distance[riderZone][zones[i]];
        int dBest = distance[riderZone][zones[best]];
        if (d < dBest || (d == dBest && ratings[i] > ratings[best])) best = i;
    }
    return best == -1 ? "NO DRIVER AVAILABLE" : names[best];
}
```

Backup: verified console output (JDK 21)

```
$ java Dispatch
Ada     | zone 3 | 4.8 | AVAILABLE
Grace   | zone 1 | 5.0 | UNAVAILABLE
Linus   | zone 4 | 4.1 | AVAILABLE
Ken     | zone 1 | 4.6 | AVAILABLE
Barbara | zone 2 | 4.9 | UNAVAILABLE
Alan    | zone 4 | 4.4 | AVAILABLE

countAvailable()   = 4
averageRating()    = 4.633333333333333
ratingOf(Grace)    = 5.0
ratingOf(Turing)   = -1.0
bestRatedAvailable = Ada

nearestAvailable(0) = Ken
nearestAvailable(1) = Ken
nearestAvailable(2) = Ada
nearestAvailable(3) = Ada
nearestAvailable(4) = Alan
```

```
$ java DispatchBoard
Dispatch board online.
4 seats, rated 4.8, zone B
Available? true
-2147483648
Ada
Grace
Linus
Barbara
Alan
Total rides today: 49
(offline)
Busiest Driver: Linus
```

⚠ The raw sum of the ratings is **27.799999999999997**, not **27.8** — binary `doubles` again. Print with `%.2f` (**4.63**) rather than comparing for equality.

Backup: error messages, word for word

```
DispatchBoard.java:1: error: class Dispatchboard is public,
                      should be declared in a file named Dispatchboard.java

Oops.java:4: error: non-static variable instanceField cannot be referenced
                    from a static context

Uninit.java:4: error: variable total might not have been initialized

Missing.java:3: error: ';' expected
        int x = 5
                ^

Sym.java:4: error: cannot find symbol
        findBusiestDriver();
        ^
  symbol:   method findBusiestDriver()
  location: class Sym

Error: Could not find or load main class dispatchboard
Caused by: java.lang.ClassNotFoundException: dispatchboard

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
        Index 3 out of bounds for length 3
        at Bounds.main(Bounds.java:5)

Exception in thread "main" java.lang.NullPointerException:
        Cannot invoke "String.length()" because "<local1>[0]" is null
```